### Document Set Overview

This package consolidates the finalized architecture into four working documents needed to begin build: a Product Requirements Document (PRD), a Technical Design Document (TDD), a System Architecture reference, and a Design System guide. All technical decisions reflect the locked hybrid architecture — Spring Boot for core product logic, FastAPI for AI orchestration, on-device inference for latency-critical tasks, and free-tier cloud APIs for heavy AI compute .

# 1. Product Requirements Document (PRD)

## 1.1 Product Vision

AI Pocket Personal Trainer is a cross-platform fitness and nutrition app that replaces a human trainer's core functions — form correction, load recommendation, program design, and diet feedback — using voice interaction, computer vision, and conversational AI, with specific accuracy for Indian/South Asian food and regional exercise needs .

## 1.2 Target User

Fitness-focused individuals, likely intermediate-to-advanced lifters, who want data-driven training feedback without a paid human coach, and who eat a predominantly Indian/regional diet that generic calorie apps misclassify.

## 1.3 Core Feature Set

### Fitness Module

| Feature | Description | Priority |
|---|---|---|
| Voice-driven logging | Log sets/reps/weight hands-free via speech, parsed on-device and structured via cloud LLM | P0 |
| Workout import | Import routines from pasted text, photo (OCR), or video (frame + pose parsing) | P0 |
| Animated exercise demos | Show correct-form animations per exercise with targeted muscle overlays | P0 |
| AI form-check | Analyze submitted lift video for joint angles, bar path, and rep velocity to flag poor form | P0 |
| Ego-lifting / load detection | Flag excessive load based on velocity loss and form breakdown; recommend weight up/down | P0 |
| Superset & workout editing | Let user request supersets or modify any planned workout via chat | P0 |
| Personalized programming | AI researches and generates a tailored program based on goals and history | P1 |
| Muscle-map injury triage | Tap a muscle region, describe pain, get a fatigue-vs-injury risk flag (not diagnostic) | P0 |

| Feature | Description | Priority |
|---|---|---|
| Conversational AI coach | Chat interface for all workout questions, modifications, and guidance | P0 |

### Nutrition Module

| Feature | Description | Priority |
|---|---|---|
| Food photo capture | Upload/take photo of plate or bowl | P0 |
| Regional dish recognition | Identify Indian/regional dishes accurately (e.g., dalma vs. generic dal) | P0 |
| User confirmation loop | Confirm/correct detected dish list before calculation | P0 |
| Portion/calorie estimation | Estimate grams and calories from image, shown as a range with manual adjustment | P0 |
| Food-quality feedback | Flag excess oil/fat visually; offer improved recipe on request | P1 |
| Combination & timing analysis | Flag poor food pairings and eating-time patterns | P1 |
| Cross-module insight | Correlate nutrition patterns with training performance in coaching responses | P1 |

### 1.4 Non-Goals (Explicitly Out of Scope for v1)

- Medical diagnosis of injuries (the app only flags risk, never diagnoses).
- Fully automatic, error-free portion estimation — outputs are always ranges with manual override.
- Support for cuisines beyond Indian/South Asian in the first release (architecture is extensible, but v1 nutrition accuracy is scoped to this region).

### 1.5 Success Metrics

- Voice logging transcription accuracy in gym-noise conditions.
- Form-check flagging precision against user-reported/coach-reviewed lifts.
- Dish recognition accuracy on regional Indian dishes (measured against the Khana Hard-20 subset benchmark) .
- Percentage of nutrition estimates requiring manual correction (target: decreasing over time as the synonym-mapping table improves).

# 2. Technical Design Document (TDD)

### 2.1 Service Ownership Matrix

| Domain | Owner | Technology |
|---|---|---|
| Auth, user profiles, workout logs, nutrition logs, sync, plans, progression rules, quotas | Core App Backend | Java Spring Boot |

| Domain | Owner | Technology |
| --- | --- | --- |
| AI gateway, provider routing, OCR cleanup, prompt assembly, embeddings, food-image orchestration | AI Orchestration | Python FastAPI |
| Unified RAG storage, user data, workout/nutrition logs, media metadata | Data Storage | PostgreSQL + pgvector + MinIO |
| Client UI, camera/mic integration, exercise animation rendering | Mobile Client | React Native |

### 2.2 Architectural Boundary Rule

FastAPI handles stateless AI processing and formatting; Spring Boot enforces all stateful product logic. This guarantees AI cannot autonomously bypass user progression rules or database integrity without passing through business validation in Spring Boot .

### 2.3 AI Inference Distribution

#### Edge (On-Device)

- **Voice**: Whisper.cpp handles continuous voice recognition on-device to mitigate gym noise and network latency .
- **Vision/Pose**: MediaPipe Pose processes real-time skeletal tracking for form and ego-lifting detection directly on the device, avoiding server-side GPU costs .
- **OCR**: Initial OCR for workout imports runs on-device first; cloud vision models are reserved strictly as a fallback for complex or degraded images that fail on-device processing .

#### Cloud API (Serverless, Free-Tier)

- Heavy LLM and advanced vision tasks route to free-tier serverless providers — Groq, OpenRouter, Hugging Face Inference Providers, and NVIDIA NIM — eliminating the need for self-hosted GPU infrastructure .
- The FastAPI gateway maintains a **priority routing / fallback matrix**: primary provider (e.g., Groq for LLM tasks, due to fastest free-tier inference) with automatic fallback to OpenRouter or Hugging Face on rate limit (HTTP 429), timeout, or degradation .
- **Background queuing**: Non-urgent AI tasks (e.g., generating a full personalized program, batch nutrition-pattern summaries) route through a quota-aware background queue so they don't compete with real-time chat/voice requests for limited free-tier quota .

### 2.4 Resilience Pattern

FastAPI implements retry, backoff, and circuit-breaker logic (via a library such as Tenacity) for all external AI calls:

- **Exponential backoff with jitter**: Failed calls to a rate-limited provider pause with increasing, randomized delay before retry.

- **Async circuit breaker**: Repeated failures past a threshold trip the circuit to an OPEN state, failing fast rather than holding dead connections.
- **Graceful degradation**: On circuit-open, a deterministic fallback (e.g., "Your workout has been saved; analysis will follow shortly") returns instead of an error, so the client never sees a crash .

### 2.5 Data Model (RAG Layer)

| Column | Type | Purpose |
|---|---|---|
| id | UUID/SERIAL | Primary key |
| content | TEXT | Raw text chunk (workout log, chat turn, IFCT nutrient profile) |
| metadata | JSONB | User ID, timestamps, document type — used for exact filtering |
| embedding | vector(768) | Semantic embedding for similarity search |

Text is chunked at 200–500 tokens with 50–100 token overlap before embedding, to preserve semantic context across boundaries . Retrieval uses HNSW indexing for the static exercise/nutrition corpus (higher recall) and IVFFlat for larger, more dynamic user-log partitions, with cosine similarity as the primary distance metric, combined with keyword/JSONB filtering in a hybrid search pattern to reduce hallucination .

### 2.6 Offline-First Sync

The mobile client uses WatermelonDB (SQLite-backed) for local caching of exercises, logs, and chat history. Sync follows a deterministic pull-push protocol: the client sends a `last_pulled_at` timestamp, the backend returns a delta payload, and local mutations are pushed as an atomic transaction, with conflict resolution via last-write-wins timestamps (or CRDTs for cumulative metrics like total training volume) .

### 2.7 Form-Check Kinematic Method

MediaPipe Pose extracts 33 three-dimensional landmarks per frame. Joint angles are derived via dot products between vectors from a central joint to adjacent landmarks; postural deviations (e.g., spinal rounding) use vector cross products against a global up-vector . Dynamic Time Warping normalizes rep duration against reference templates, and Normalized Cross-Correlation scores bar-path similarity; a sharp drop in concentric velocity combined with form deviation triggers the ego-lifting heuristic and downward load recommendation .

### 2.8 Nutrition Pipeline Method

1. Image capture → segmentation (SAM 2, hosted via cloud inference) isolates each food item and its container .
2. Depth estimation (Depth Anything V2, hosted via cloud inference) reconstructs relative 3D surface geometry .
3. Classification (ViT-based model, cloud-hosted) identifies dishes; ViT is selected over CNN baselines due to substantially higher accuracy on the Khana "Hard-20" subset (90.7% vs.

42.0% for EfficientNet-B0) .

4. Regional-name resolution maps recognized classes to the closest IFCT 2017 entry via a maintained synonym table (e.g., dalma → dal category), expanded through user corrections .
5. User confirms/edits the candidate list via chat UI before calculation.
6. Scale calibration (reference object or saved plate diameter) converts relative depth to absolute volume, then a food-class density factor converts volume to grams .
7. Nutrient values are queried from IFCT 2017; results are shown as a range with a manual gram-adjustment slider, since single-image volume estimation is an inherently ill-posed scale problem .
8. Quality reasoning (oil/fat visual cues, combination checks, meal timing) runs via the LLM, grounded in the confirmed dish's nutrient profile .

# 3. System Architecture

## 3.1 High-Level Topology

The architecture separates four layers: client (React Native), backend (Spring Boot + FastAPI), data (PostgreSQL/pgvector + MinIO), and external services (cloud AI providers, wger, IFCT 2017) . The mobile client talks to Spring Boot directly for auth/logs/sync, and to FastAPI for anything requiring AI processing; FastAPI in turn hands validated, structured results back to Spring Boot for persistence, ensuring AI never writes directly to the source-of-truth database without business-rule validation .

## 3.2 Voice Logging Data Flow

Voice input is transcribed on-device via Whisper.cpp, sent to the FastAPI gateway for structuring into JSON via a cloud LLM, then validated and persisted through Spring Boot .

## 3.3 Nutrition Logging Data Flow

Food photos are processed through segmentation, classification, and depth estimation on cloud APIs, presented to the user for confirmation, converted to a nutrient estimate against IFCT 2017, and enriched with LLM-generated quality feedback before being persisted as a log entry .

## 3.4 External Data Dependencies

| Source | Role | Notes |
|---|---|---|
| wger REST API | Exercise taxonomy, equipment, muscle-group SVGs | Self-hosted instance recommended for control and offline caching |
| IFCT 2017 | Indian nutrient composition database (528 foods) | Published by NIN/ICMR; used as authoritative nutrient source |
| Khana dataset | Indian food image classification training/benchmark data | 131K images, 80 labels, used to validate/fine-tune vision models |

| Source | Role | Notes |
|---|---|---|
| IndianFoodNet | Supplementary Indian dish image dataset | 30 dish classes, ~5,500 images |

### 3.5 Provider Fallback Matrix (AI Tasks)

| Task | Primary | Fallback 1 | Fallback 2 |
|---|---|---|---|
| Chat/LLM reasoning | Groq (Llama 3.3/4) | OpenRouter free pool | Hugging Face Inference Providers |
| Speech-to-text | On-device Whisper.cpp | — (no cloud fallback needed) | — |
| Food vision classification | Hugging Face Inference Providers / Gemini free tier | NVIDIA NIM | OpenRouter multimodal |
| Segmentation (SAM 2) | Hugging Face / Cloudflare Workers AI | — | — |
| Depth estimation (DA-V2) | Hugging Face | — | — |
| Embeddings | Cloud embedding API (e.g., Gemini text-embedding-004) | Hugging Face free embedding endpoint | — |

This table operationalizes the "priority routing matrix" requirement from the locked architecture spec, giving explicit primary/fallback order per task .

# 4. Design System

### 4.1 Navigation Structure

Tab-based navigation across six primary sections: Workout, Chat, Progress, Muscle Map, Nutrition, Profile — chosen for cognitive simplicity during physical exertion .

### 4.2 Core UI Patterns

| Pattern | Used In | Behavior |
|---|---|---|
| Confirmation chat card | Workout import, food recognition | Shows AI-detected items as an editable list; user taps to confirm/correct before data is persisted |
| Range-based estimate display | Calorie/portion results | Never shows a single false-precision number; always a range plus a manual slider |
| Interactive muscle map (SVG) | Injury triage, exercise targeting | Tappable front/back body diagram; tap opens a pain-descriptor form |
| Voice capture indicator | Workout logging | Persistent mic button with live partial-transcript overlay during recording |
| Animated form demo | Exercise detail view | Looping reference animation with overlaid targeted-muscle highlight |

### 4.3 Visual Language Principles

- High-contrast, large-touch-target UI for use mid-workout (sweaty hands, quick glances).
- Muscle-map and exercise-demo visuals reuse wger's existing SVG/anatomical assets to avoid custom asset production for v1 .
- Nutrition confirmation screens use a conversational, low-friction tone ("I see dalma, rice, and bhindi sabzi — is this right?") rather than a rigid form, reducing correction fatigue during regional dish onboarding.
- Feedback from the AI coach (form corrections, nutrition flags) is visually distinguished from raw data logs, so users can tell "AI observation" apart from "confirmed fact."

### 4.4 Accessibility & Trust Signals

- All AI-derived health/injury guidance is labeled as a risk-flag, not a diagnosis, directly in the UI copy, consistent with the architecture's framing of the triage engine .
- Every AI-detected value (food item, exercise, calorie count) is editable inline — no AI output is ever presented as final without a user-accessible correction path.
</content>